

Guia de Colaboração Git & Arquitetura SISGED

Manual de Boas Práticas, Fluxo de Branches e Separação de Camadas (MVC)

1. Estrutura e Arquitetura do Projeto (MVC)

O projeto segue obrigatoriamente o padrão **MVC (Model-View-Controller)** para garantir a separação clara de responsabilidades e evitar a mistura de HTML, PHP e SQL no mesmo arquivo.

```
SISGED/
├── app/                                # BACK-END E REGRAS DE NEGÓCIO
│   ├── config/                        # Conexão com o Banco de Dados (database.php)
│   ├── controllers/                  # [Dev Back-end] Regras de Negócio e Validações PHP
│   └── models/                      # [DBA / MySQL] Consultas SQL e Execução PDO
├── database/                          # [DBA / MySQL] MODELAGEM E BANCO DE DADOS
│   ├── der_mer/                     # Diagramas ER (Conceitual, Lógico, Físico)
│   └── schema.sql                   # Script DDL de criação das tabelas
├── public/                           # FRONT-END (Acesso público no navegador)
│   ├── assets/                      # Imagens, logotipos e ícones (Identidade FIEMG)
│   ├── css/                        # [UI/UX] Estilos CSS (Mobile First)
│   ├── js/                         # [Dev JS] Lógica de tela e requisições Fetch/AJAX
│   └── views/                       # [UI/UX & Dev JS] Interfaces HTML sem PHP/SQL
├── docs/                            # Documentação, Matriz RACI e Contrato de Time
├── index.php                        # Roteador Principal / Entry Point
└── README.md                       # Informações gerais do repositório
```

2. Divisão de Responsabilidades e Pastas

Cada integrante da equipe deve trabalhar estritamente nos arquivos sob sua responsabilidade:

Função / Integrante	Diretórios Autorizados	Responsabilidades Técnicas
 UI/UX e HTML/CSS	public/views/ public/css/ public/assets/	Criar interfaces HTML responsivas (Mobile First), estilização CSS (identidade FIEMG) e garantir os IDs mapeados para manipulação no JS.
 JS / Integração	public/js/	Capturar formulários e botões por ID, efetuar validações no cliente, enviar dados via <code>fetch()</code> em JSON e atualizar a tela.
 Back-end (PHP)	app/ controllers/ app/config/	Receber JSON do JS, validar sessão/permissão, processar lógica de negócio e chamar o Model. <i>Proibido escrever SQL aqui.</i>
 DBA / MySQL	app/models/ database/	Criar diagramas ER, scripts SQL e classes Model com consultas SQL protegidas contra SQL Injection usando PDO.

3. Passo a Passo Inicial: Atualizar a Máquina Local

Antes de iniciar qualquer tarefa na sua máquina, sincronize o seu projeto com a branch `main`:

```
# 1. Mude para a branch main
git checkout main

# 2. Baixe as atualizações da nuvem
git pull origin main
```

4. Branches Individuais por Papel

REGRA OBRIGATÓRIA: NUNCA faça commits diretos na branch `main`.

Cada integrante deve utilizar sua respectiva branch:

- **UI/UX e HTML/CSS:** `feature/ui-ux-design`
- **JS / Integração Front-end:** `feature/frontend-javascript`
- **Lógica Back-end (PHP):** `feature/backend-php`
- **DBA / MySQL:** `feature/database-models`

Para alternar para a sua branch (ou criá-la pela primeira vez):

```
# Para criar e mudar para a sua branch (pela 1ª vez):
git checkout -b feature/sua-funcao-aqui

# Se a branch já existir no seu PC:
git checkout feature/sua-funcao-aqui
```

5. Salvando e Enviando seu Trabalho (Commit & Push)

1. Verifique o status das suas alterações: `git status`
2. Adicione os arquivos alterados: `git add .`
3. Crie o commit formatado: `git commit -m "tipo: descricao da tarefa"`
Exemplos: feat: cria tela de cadastro de turmas | fix: ajusta query de busca de instrutores
4. Envie sua branch para o GitHub: `git push origin feature/sua-funcao-aqui`

6. Unindo o Código com a Main (Pull Request)

1. Acesse o repositório no **GitHub**.
2. Clique no botão **"Compare & pull request"**.
3. Confirme se a origem é a sua branch e o destino é a `main`.
4. Descreva brevemente o que foi implementado e solicite a revisão de um colega.
5. Após aprovação, faça o **Merge pull request**.

Dica para a Defesa Individual: O professor avaliará a separação das camadas. Certifique-se de que nenhum SQL esteja no HTML ou no Controller, e que todas as consultas passem pelo arquivo Model correspondente.